

Secure Software Development

Learn, Understand, Gain Insight, and Become Aware



DIGITAL MAELSTROM

Contents

Chapter One: Learn	3
Chapter Two: Understand	10
Chapter Three: Gain Insight	16
Chapter Four: Become Aware	25
Next Steps	36
Endnotes	37

Chapter One: Learn

Learn the Reasons to Utilize Secure Software Development

In 2018, Marriott International announced that their company was subject to one of the [largest data breaches in recent history](#)¹, with 500 million customers' information being stolen after a large-scale hack. The company's earnings lowered significantly as a result, and hundreds of millions of customers lost their trust in Marriott.

This data maelstrom has left many wondering: what went wrong?

An answer might be traced back to a fundamental issue with Marriott International's website: a lack of secure development. Unfortunately, companies will create their websites without implementing safeguards to deter cyber thieves. These companies are unknowingly allowing their customer's personal information to be stolen, deleted, or even extorted for financial gain, rather than taking the steps necessary to provide their customers a safe experience on their site.

Luckily, there is a practice that can be utilized by companies to help minimize the risk of malicious cyber-activity: Secure Software Development (SSD). The goal of this book is to help you learn what SSD is and why it matters, understand what differentiates this process from others, gain insight into what occurs during each stage of the cycle, and become aware of the vulnerabilities to software.

IS IT NECESSARY?



When considering whether a company should utilize any paid service, you should ask: "Is this necessary?"

In the case of SSD, the most common answer is: "It depends."

Yet in today's complex software landscape it is most likely that the eventual answer is a "yes".

Companies often have a regulatory or contractual compliance to ensure information is kept private. This is most frequently seen in industries such as financial services, technology, financial technology, government, healthcare, and insurance.

The next section outlines many of the regulations that affect these industries. However, there are reasons beyond the law that strongly encourage businesses to use SSD: reputational damage, loss of investor trust, and others.

Regulatory and Compliance Requirements for SSD

Millions of companies are subject to government regulations that require SSD. Without abiding by these laws and regulations, businesses may be unable to carry out operations for their customers. The following are several of the security regulations that businesses must be aware of and diligently follow.

Payment Card Industry and Data Security Standard ([PCI-DSS](#)²)

All companies that accept, transmit, store, or process credit cards are required to uphold a secure environment for the information.

NYDFS Cybersecurity Regulation ([23 NYCRR 500](#)³)

Regulations by the New York Department of Financial Services that require financial institutions to abide by cybersecurity requirements.

Gramm Leach Bliley Act ([GLBA](#)⁴)

Financial institutions are required to safeguard sensitive data and to explain their practices of information-sharing to their customers.



Family Educational Rights and Privacy Act ([FERPA](#)⁵)

All schools that receive funds under an applicable program of the U.S. Department of Education are required to uphold the privacy of all students' education records.

Health Insurance Portability and Accountability Act of 1996 ([HIPAA](#)⁶) Privacy Rule

Government programs that pay for healthcare must maintain the privacy of patients' Protected Health Information (PHI).

The Federal Information Security Modernization Act ([FISMA](#)⁷)

Federal agencies are required to document, develop, and utilize a program designed to increase information security and protection.



California Consumer Privacy Act of 2018 ([AB 375](#)⁸)

Many for-profit companies which complete business in California must keep their customers' data secure, have transparency about their data collection and usage practices, and give customers the right to have their information forgotten by the company.

EU General Data Protection Regulation ([GDPR](#)⁹)

Companies that have access to citizens of the European Union's data must abide by a set of regulations in order to safeguard the movement and processing of individuals' personal data.

Directive on Security of Network and Information Systems ([NIS Directive](#)¹⁰)

Providers of essential services which complete business in the European Union must follow a set of regulations which strive to implement a standard of information and network security.

[Sarbanes Oxley Act of 2002](#)¹¹

Corporate executives are required to prove the accuracy of their company's financial statements, assess and maintain internal controls in order to prevent inaccurate, fraudulent, or misleading data about company finances, all upon penalty of law.



As illustrated by this list, a vast array of industries fall subject to the legal requirements of protecting clients' personal information. Not only will following these regulations prevent businesses from becoming tangled in trouble with the law, customers will appreciate the effort given to protect their information and they will place their trust into the business.

Additional Reasons to Consider Secure Software Development

Maintaining compliance is a reason to invest in Secure Software Development. But if the laws weren't in place/aren't definitively applicable to some industries, why should a company utilize this service? It turns out that businesses that make their consumers' security a top priority greatly benefit from this decision. The following reasons are several of the most prominent factors that may be positively influenced by choosing Secure Software Development.

Access to Insurance

Data breaches are such a detrimental and costly issue in recent years that many insurance companies have begun addressing the matter in their policies. Many modern business umbrella policies and cyber insurance policies state that they will cover data loss, poorly functioning equipment, loss of business due to IT systems being down, etc. However, without being able to provide evidence of good practice, cybersecurity insurance rates may be much higher. Insurance companies do not have to provide coverage to software companies that do not adhere to the standards of due diligence in preventing cyber attacks during the writing stage of software development.

Reducing Liability

When data breaches occur, all parties involved suffer. The organization holding the personal information can end up paying substantial amounts in damages to customers affected by the breach. Courts assign [liability](#)¹² to an organization if they did not implement reasonable safety measures or follow safety standards required by the law, if the organization did not mitigate/remedy the damages after a data breach, or if they failed to inform the affected consumers in a timely manner. A wise business owner's goal is to reduce liability in the case of a data breach.

Positive Consumer Opinions

Ask somebody what they thought of Equifax after the company exposed the sensitive information of 147 million users in what is one of the [largest data breaches in history](#)¹³, or their thoughts when the names, passwords, addresses, and phone numbers of all three billion Yahoo users was [hacked](#)¹⁴ in 2013, and one will likely receive some strongly-worded responses. To learn more about how widespread of an issue data breaches are, visit the site [haveibeenpwned.com](#) to see every instance of breach in which your personal information has been compromised. People shudder at the notion that hackers could access their personal information through an organization that they use and trust. Consumer trust is everything; once it is broken, it will take great effort to rebuild. The mantra "with great power, comes great responsibility" serves as an effective illustration for organizations holding their consumers' information.

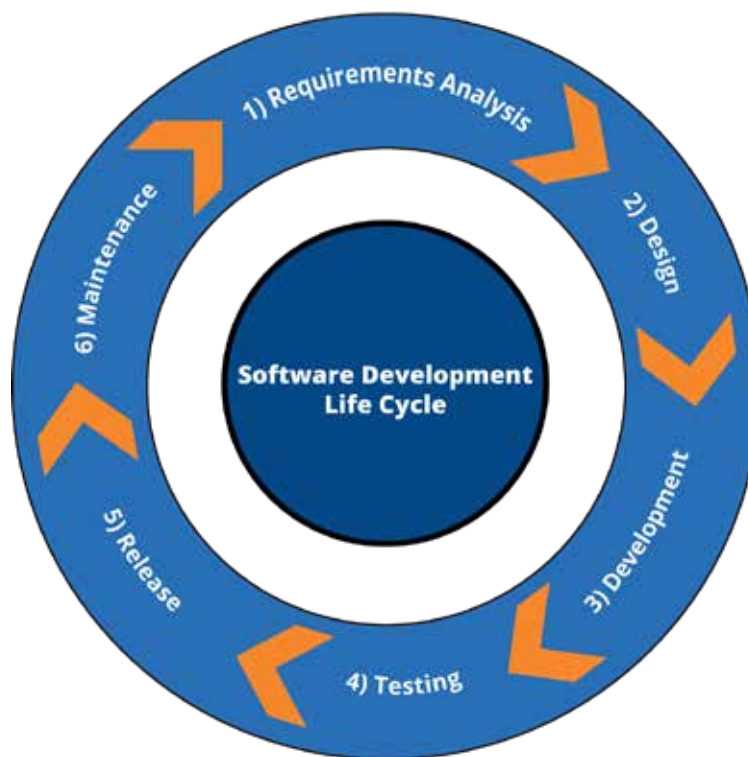
Financial Gain

Undoubtedly one of the most important factors to a business is its financial capital. The element that defines a business as a business. If large-enough hindrances to the production and retention of financial value exist, a company has the potential to go bankrupt or suffer other consequences. The statistics behind business failure in regards to data breaches are alarmingly high. The Ponemon Institute stated in a [2017 report](#)¹⁵ that the average cost of a data breach for businesses in 2017 was \$7.35 million. If company software was developed with security in mind, the amount of overall business success would increase substantially.

Chapter Two: Understand The Differences

Understand the Differences Between Software Development As You Know It and Secure Software Development

After considering the legal requirements and the practical reasons to consider utilizing SSD, a second question must be answered: what are the differences between the basic version of the software development life cycle (SDLC) and the Secure Software Development Life Cycle (SSDLC)? Those who have owned a company with an online presence have likely undergone the regular process of developing their own software. Typically the SDLC process follows a general model:



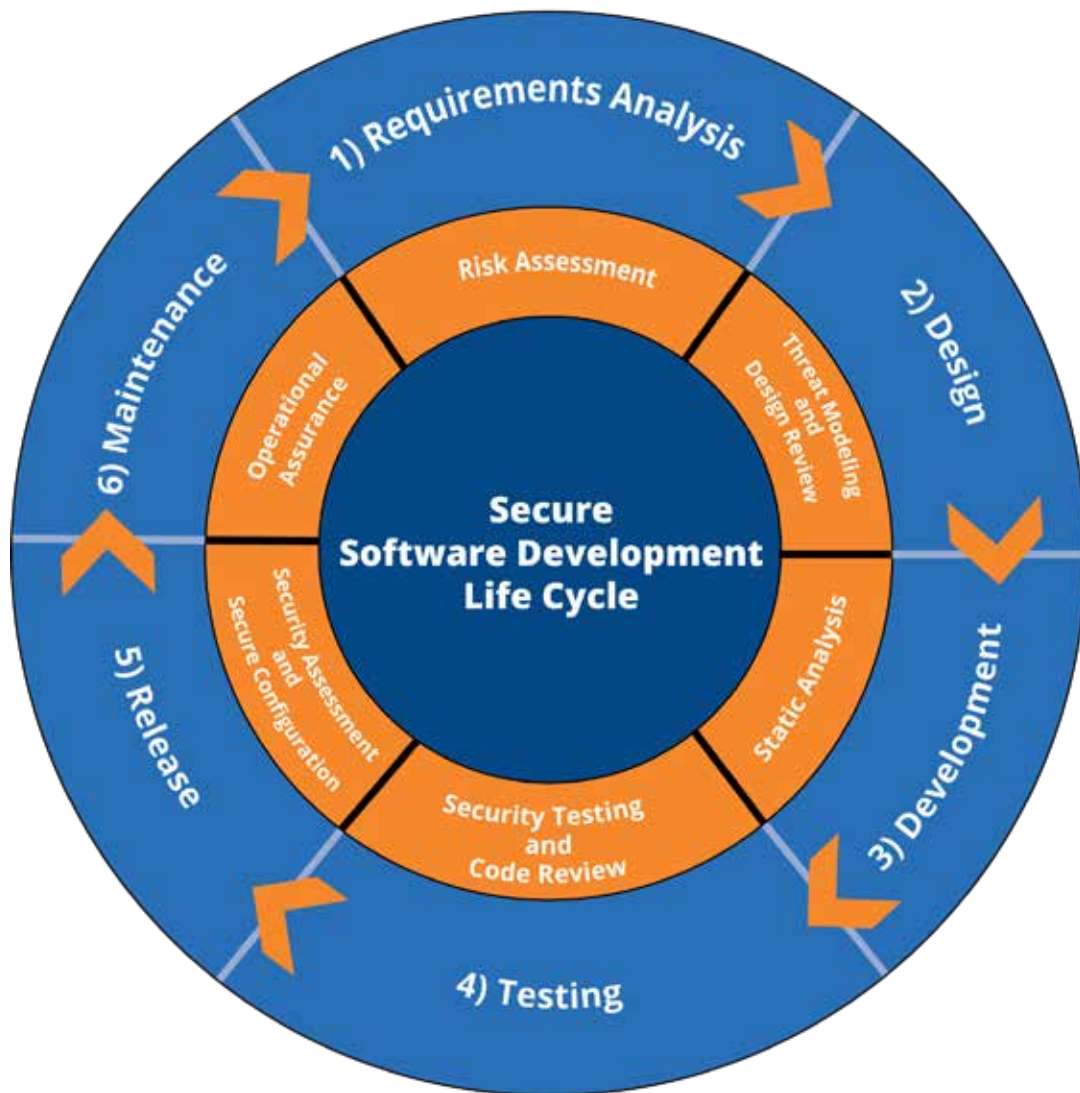
1. First, the engineers compile a set of requirements for the company's software.
2. Next, developers will take these requirements into account and begin designing the scope of the software.
3. The actual development stage of the software is then begun by the engineers.
4. After the software has been developed and it has been approved by the client, the developers must test it in varying capacities for quality assurance and to ensure there are no major code issues.
5. The software is then released on the client's server and can be accessed by users.
6. After its release, the developers maintain the usability of the software and fix minor bugs that may arise.
7. The last stage of this cycle is retirement/decommission. This stage occurs whenever it has been decided that the software will no longer be used. The operators disable it and retire or archive its data.



In theory, this process is passable for a company to utilize if their software does not manage any confidential information (that is, information required to be protected). On the other hand, this cycle would be unfit for companies that store sensitive information (information that is recommended to be protected) in any capacity. The process of SSD follows the

steps of general software development while simultaneously interweaving security checkpoints into each stage to ensure that the end product is secure and high-quality upon deployment.

When comparing the two processes at face-value, SSD requires more hours of work and resources than software development without a security component. Fortunately, the extra time and expense invested into SSD are crucial in avoiding or limiting the impact of the inevitable data breaches and other security-related problems accompanying unsecure software. The Secure Software Development Life Cycle (SSDLC) follows this model:



Risk Assessment (and Requirements Analysis)

The first step of the SSDLC is Risk Assessment. During this step, a group lead by specialists and composed of both developers and the business and/or data owners will identify the potential risks associated with the software. This step is completed in tandem with the Requirements Analysis stage of the standard software development life cycle (SDLC). Conversely to SSDLC, the SDLC does not include any steps pertaining to identification and mitigation of security risk during its Requirements Analysis phase. Risk assessment, along with the other stages of the SSDLC, is subject to be an ongoing process within the cycle to allow changes to be made to the software and to be completed again at a regular cadence to help illustrate new or changed risks that become apparent.

Threat Modeling and Design Review (and Design)

The group of specialists, developers, and business owners/data owners will then define the minimum security criteria that should be implemented throughout the process. A structured approach is used to identify threats, mitigate those threats, and then ensure that they have been properly mitigated. This step, known as Threat Modeling, gives the development team the opportunity to discuss the security of their current software amongst themselves and security-focused peers. Next, developers will utilize several security features to fulfill the secure design requirements in the Design Review stage. Security and encryption standards will be designed and implemented, as well as the more basic software elements that are completed during the Design phase of the SDLC.

Secure Implementation (and Development)

During the Secure Implementation phase, the engineers will consider the security risks associated with utilizing third party code - such as libraries and frameworks - and prepare to mitigate these potential risks. Developers may use tools such as static analysis tools or other security tools which have been approved for use in the software construction process. These tools will be listed along with any necessary configuration for secure operation.

With the approved tools and secure practice guides, the developers then take part in Secure Implementation. This is the phase where developers use their resources to write high-quality, secure code. At this stage, the Development phase of the SDLC occurs and the developers begin creating the software.

Security Testing and Design Review (and Testing)

At the Security Testing and Design Review stage, a series of tests will be performed on the software to validate the effectiveness of its security controls: a test on units of functionality (also known as unit testing) as an additional measure to prevent mistakes, a test on the sum of the software's components (also referred to as integration testing), and a test in which the developers act as hackers and attempt to breach the software by using tactics that an authentic hacker would use (also known as penetration testing). Instead of testing only for quality assurance and ensuring there are no major code issues (as would occur in the Testing phase of the SDLC), security is a major component of the tests.

Security Assessment, Attack Surface Reduction, and Secure Configuration (and Release)

During the Security Assessment of the SSDLC, developers run further validation tests on the software to ensure that it is ready for release. At this point, the developers analyze the secure software project as a whole and define which components can undergo further securing.

The developers follow another security measure known as Attack Surface Reduction. In this stage, the development team assesses the whole of the software, looking for areas in which the software is susceptible to attacks from external sources. The security architects use this insight to effectively minimize the attack surface of the software.

Finally, the developers have reached the Secure Configuration phase. The finishing touches are added to the software to ensure it remains secure during and after it is released. Developers configure security-focused infrastructure for the software, and the Release stage of the SDLC is finally reached. If the developers have successfully completed the phases of both the SDLC and the SSDLC, users are now able to access the software and interact with it securely and productively.

Operational Assurance (and Maintenance)

As the software operates, the developers consistently engage in Operational Assurance: that is, running tests and analyzing the application to ensure the software remains secure and that there are no vulnerabilities. If and when vulnerabilities become known over time, the SSDLC continues its cycle of security steps to mitigate the potential issues. This step occurs jointly with the general Maintenance phase of the SDLC.

Decommission/Retirement

A step not explicitly stated in either of the two software life cycles - yet is still important to explain - is the Decommission/Retirement phase of the software's life. When a stakeholder decides that the software should no longer be in use, the developers may remove the application from production or decommission the system entirely. The software may be retired because the release is no longer supported, the software is being replaced by another system, the system has become obsolete, or for a myriad of other reasons. This phase may occur at the end of both the SDLC and the SSDLC.

Chapter Three: Gain Insight

Gain Insight Into How Secure Software Development Protects Your Custom Software

At this stage, a third question must be answered: how exactly does the SSDLC work? In other words, what are the specific tasks Secure Software developers are completing for the company that is investing in their service?

It is essential to remember that the SSDLC includes all of the steps of the standard SDLC, with a component of additional steps in place to increase security measures. Both processes are continuous and each last from the first conception of a software product until the day it is retired from service.

risk. When a business accepts the risk, a requirement will be included wherein the business regularly reevaluates the risk in order to see whether or not the risk grows as the software grows and if it must be handled differently in the future.

Risk Assessment

The goals of the first stage of the SSDLC are to compile an explicit inventory of all of the potential risks to the data, and then to devise a plan for mitigating each of those risks. A group consisting of the developers, business stakeholders, security experts, lawyers, and possibly customers and vendors will work to identify the risks and explain the harm that each could cause. The business then has several options as to how they may address the risk. The decision-makers may choose to make changes to the software's behavior in order to mitigate the risk; the risk may be transferred to another organization (such as a service provider) and spelled out in contractual terms; or most frequently, the business may decide that the cost to address the

risk is greater than the expected harm of the risk and will choose to simply accept the risk. When a business accepts the risk, a requirement will be included wherein the business regularly reevaluates the risk in order to see whether or not the risk grows as the software grows and if it must be handled differently in the future.

After the company has decided which potential issues they would like to have handled, the software stakeholders and developers will then jointly make decisions on how to mitigate each of them in the most efficient and effective methods possible.

To give more insight into what the risk assessment process may look like, the National Institute of Standards and Technology (NIST) published a national standard in 2012 titled Special Publication 800-30: Guide for Conducting Risk Assessments. To summarize the process described in the nearly 40 page publication, the following is a greatly condensed description of the Risk Assessment process.



Step 1: Prepare for Assessment

- Identify Purpose of Risk Assessment
- Identify Scope of Assessment
- Identify Assumptions and Constraints
- Identify Sources of Information for Threats, Vulnerabilities, Impacts
- Define Risk Model, Assessment Approach, and Analysis Approach

Step 2: Conduct Assessment

- Identify Threat Sources and Events
- Identify Vulnerabilities and Predisposing Conditions
- Determine Likelihood of Occurrence
- Determine Magnitude of Impact
- Determine Risk

Step 3: Communicate Results

Step 4: Maintain Assessment

To read a much more in-depth description of the risk assessment process, Chapter 2 covers the core stages of the process, key terms, concepts, and more.

Keep in mind that the list of potential risks is a living document. As new potential risks develop and become evident through each stage of the process, they are added into the existing compilation of risks and handled accordingly. Several of the most common risk factors to software are defined and explained in the next section and in Chapter Four.

Threat Modeling and Design Review

After risks are assessed, a team of threat modelers (developers, engineers, stakeholders, decision-makers, and managers) will conduct the Threat Modeling stage of the SSDLC. The team of security professionals will take on the role of an attacker and imagine well-known/well-documented attacks against the software. This is done in order to discover and prioritize areas that need security attention first. This gives the development team insight as to which steps they must take to prevent a real attack from succeeding. In essence, the team mentally stages a theoretical hacking. Many secure software developers follow the Microsoft [STRIDE Model](#)¹⁶ list of common cyber threats when threat modeling:

(S)poofing - an attacker falsely poses as something/someone else.

- Example: a malicious email is created to look like it was created by a company, and it is sent out to employees of the company in hopes of collecting their data.

(T)ampering - an attacker changes software's code or system data.

- Example: somebody hacks into a company's application and changes the code in a way that makes the data inaccessible.

(R)epudiation - an attacker carries out an attack on an application in a way that does not track their steps, therefore allowing them to deny they were responsible for the action.

- Example: a customer purchases a pair of shoes on a clothing website, and confirms their transaction on the checkout page. An attacker deletes the transaction history, resulting in the site failing to display that the customer paid for their item.

(I)nformation Disclosure - an attacker reveals private information that is not intended to be revealed.

- Example: a hacker uses a series of actions to gain access into a hospital's online database of confidential patient information.

(D)enial of Service - a situation in which an attacker uses malicious software (malware) to prevent customers from accessing a company's website; resulting in loss of company revenue or other business activities.

- Example: a very large amount of unauthorized bots attempt to simultaneously access a company's website. The company's on-premise server that is responsible for directing the traffic to the website becomes overwhelmed by the velocity, and is unable to direct any traffic to the website. Nobody can access the site because the server is unresponsive.

(E)levation of Privilege - an attacker gains unauthorized high-level power of an application.

- Example: a user discovers a vulnerability within a food delivery mobile application which them to gain the operational capabilities of an administrative role. The user can then take administrative actions such as viewing user's addresses, changing food orders, viewing prior transactions, etc.

There are several reasons why developers complete the Threat Modeling stage. This process offers an early opportunity to identify potential security problems in the design phase, and it tends to identify categories of potential vulnerabilities (as shown above). Many software developers follow Microsoft's STRIDE Model of Threats to identify the most common types of threats (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege).

The second facet of this stage is Design Review. The developers construct the software with a high emphasis on mitigating threats. Developers aim for a concept referred to as defense-in-depth¹⁷: secure design and secure implementation. Defense-in-depth is the idea that developers layer many kinds of security controls

so that if an attacker succeeds in defeating the first layer, they are further challenged by new layers of protection. This makes the cost of attacking a system much higher for attackers in terms of both time and money. newly-designed software is then reviewed by developers to ensure it retains its secure posture after it has been designed as it prepares for operation.

Static Analysis

The third stage of Secure SDLC involves an analysis of static (non-running) software. Developers run their own tools/software within the software that is actually being created for the business to look for vulnerabilities in the code (the computer's language). Using these Static Code Analysis tools/software programs is an industry standard practice - deeming them a helpful instrument to use in addition to human developers. Digital Maelstrom regularly uses reputable Static Code Analysis tools/software such as [SonarQube™¹⁸](#), [VeraCode](#), and [Code Climate](#).

Security Testing and Code Review

The fourth stage of the SSDLC is dedicated to finalizing the software and ensuring it is as secure as possible before it is made available to users. The first half of this stage, security testing, is characterized by developers writing security-focused automated tests and manual tests for the software. These tests help ensure that the software not only behaves as expected, but behaves properly when it is installed and configured for real use.

Companies that opt for SSD in addition to a well-implemented security program will have business requirements fulfilled by the developers as well as an extra level of security requirements and testing.

The second half of this stage is code review. After the software has undergone automated tests and analyses by other software and tools, a human developer takes the time to review the source code of the software. This is arguably the most powerful method for spotting security issues because humans are good at seeing broad and narrow patterns in context to the software's purpose where machines tend to only identify a more narrow set of patterns.



Security Assessment and Secure Configuration

What is Encryption?

The process of encryption consists of encoding a message so that only the users who are allowed to access that information are able to open it up and read it. Encrypted messages appear as a mess of random numbers and symbols to somebody who does not have access to the encryption key, making this process incredibly useful for hiding sensitive information from potentially unauthorized and/or dangerous users alike.

The first half of this step is Security Assessment. During this stage, developers complete the real validation testing of the entire software to ensure that it is ready to be used in practice. Developers run a variety of all-encompassing tests to ensure that there are no issues with the software, and if any are caught then they can be fixed.

After security assessment is completed, the developers follow the very last step- secure configuration. In this stage, the developers add several finalizing elements of security to the software so that it can be made available for users and can enter its operating state without losing any security tuning. This phase configures all the complex parameters of cryptographic algorithms and ensures that unsafe forms are not allowed.

Developers might also use encryption during Secure Configuration. Businesses may use encryption in multiple places throughout a software to keep private configuration values safe from unauthorized disclosure or abuse: including database configurations (the way a collection of data is organized) and configuration files (customizable attributes that define how an application is controlled).

In the last piece of secure configuration, developers configure security-focused infrastructure for the software. Examples of what developers may deploy and add to the software are:

- a Web Application Firewall (WAF) - a tool that acts as a shield against online activity that may be attempting to bombard a website with malicious/potentially dangerous data.
- Load Balancer - a tool that has been configured by a human to divide and direct network traffic to its resource server to ensure that no server is overloaded with too many requests.
- Content Delivery Network (CDN) - a tool that provides fast delivery of content, HTML, and its supported files (pictures, videos, and stylistic components). It is often deployed with Web Application Firewall (WAF) aspects and serves to isolate application-hosting computers from needing to talk to the wide-area Internet directly.



Each and every step of the SSDLC coincides with each other in order to create the most secure program possible. Rather than following a linear process, the SSDLC is circular in that it is a continual progression. Steps are subject to be repeated multiple times to ensure the highest level of security for the software. The next section of this book consists of a compilation of the vulnerabilities and potential risks that may affect software.

Chapter Four: Become Aware

Become Aware of the Array of Vulnerabilities and Attacks That Affect the Software

The purpose of the last chapter of this informational booklet is to lead you to become aware of the multitude of vulnerabilities and attacks that have the potential to negatively affect the software. While these two terms may sound very similar, there is a difference in how they are classified. A vulnerability is a property of a system which allows it to be attacked in some form. An attack is the negative outcome of a vulnerability: a specific set of steps which allow someone to do something they otherwise shouldn't be allowed to do, or to use the application in a way they shouldn't be able to use.

In conjunction with [OWASP Top Ten Items of 2017](#)¹⁹, Digital Maelstrom has compiled a list of the most common threats to software.

Security Misconfiguration

[Security misconfiguration](#)²⁰ is a vulnerability that is created whenever a human developer, administrator, or end-user makes an error when editing or changing configuration values within the software. This results in the software becoming potentially insecure. Common examples of this include: typos in configuration directives within configuration files, incorrectly or inappropriately setting configuration values in a way that reduces security, misunderstanding the role of a configuration setting in the context of a larger application, or otherwise failing to complete important steps during configuration. Each of these examples can lower the software's security in some shape or form.

A real-life example of a [security misconfiguration incident](#)²¹ occurred in 2015 with the popular crowdfunding company Patreon. Security researchers discovered that a debug version of the Patreon web application was configured in a way that opened a console whenever the system found an error within the code. After the console was opened, anybody had the opportunity to inject code directly into the Patreon app. A hacker noticed this vulnerability and accessed the company's database of customer information - exposing users' full names, emails, addresses, and posts. Unfortunately, security misconfiguration vulnerabilities such as Patreon's are often overlooked and result in attacks on company software.

Secure Software Development mitigates this risk by providing multiple levels of safety nets that catch these errors on the off chance they are configured. Developers can significantly lower the risk of security misconfiguration by writing software in a way that uses secure default configuration, creating software that warns administrators when certain configuration values have reduced the security of the system, and including automated security tests.



Injection Attacks

Injection Attacks occur whenever an attacker sends a harmful command disguised as a piece of data to an application. These attacks are successful whenever the attacked application interprets the false command as authentic. The implications of a successful attack could be corruption, data loss, access requests being denied, or ultimately a complete takeover of the application by the attacker. These types of attacks are relatively common.

With Secure Software Development in place, these types of attacks can be prevented through various security controls such as whitelisting (allowing specific users to access some aspects of a software), input validation (ensuring that entered data is useful and correct), and other approaches. Developers use software scanners to constantly review the data to discover if there are any injection flaws (areas where, if attacked, injection attacks would be successful). Luckily, injection flaws can typically be addressed, fixed, or weakened by layering security controls. Several common types of injection attacks include:

XSS

Cross Site Scripting ([XSS](#)²²) - An attacker injects malicious data into a web page which is not intended by the developer which is then run by unsuspecting user's browser.



[Code Injection](#)²³ - An attacker injects malicious code into an application's existing code to create computer bugs that will significantly hurt the software.



Lightweight Directory Access Protocol ([LDAP](#)²⁵) Injection - An attacker injects malicious code into queries of an application's directory of users - allowing them to access and change user data.



Structured Query Language ([SQL](#)²⁶) Injection - An injection targeting an application's database of information. This attack is disguised as query language and is meant to view, change, or destroy data. Many applications store all of their data in SQL databases, making a successful SQL Injection particularly detrimental.



[Script Injection](#)²⁴ - An attacker pretending to be a user injects malicious code into the user interface elements of a web application, like a form that is intended to be filled out by users, in hopes of accessing the database to access or change the site's information.

Phishing Scams



This attack may be a bit more familiar than other attacks explained in this section. A Phishing Scam occurs whenever an attacker disguises themselves as a trustworthy entity in an attempt to trick users into giving them their personal information. These attacks often occur by means of email, social networking, phone, or an

instant messaging platform. Phishing is as much a technical attack as it is a social engineering tactic. The basic premise of phishing is to convince a human to do something they normally would not do.

Phishing Scams hide behind the illusion of legitimacy to help achieve the attacker's goal of enticing a user to perform an operation that causes typical built-in defenses to fail. These operations can be actions as simple as clicking a link in a browser or an email, opening a "trojaned" attachment, etc).

Consider this scenario: An attacker successfully hacks into a company's trusted software platform after it was developed with little security in mind. The attacker crafts an email, making it appear as if it was issued from an internal company platform, tool, or software. The attacker sends the phishing email to the employees of the company. When the employees see the email, which appears to have been sent by their trusted employer, they click on the attachment and unknowingly give the attacker even more information about the business. Attacks such as these have the potential to lead to a host of additional cyberattacks.

Phishing is best thwarted through a combination of employee security training and developing company software with the SSDLC to ensure the trusted software cannot be leveraged against its users. If done well, these practices will result in greatly reduced chances of users being phished.

Brute Force

A Brute Force Password Guessing Attack is defined by an attacker bombarding an application's login form with countless combinations of passwords in an attempt to break into a user's account. These attacks can occur online by attacking the login page or form, or offline by bombarding a database with encrypted combinations of passwords retrieved from a database or an application.

Many applications already have safeguards in place to prevent these attacks from being successful such as two-factor authentication, Completely Automated Public Turing Tests To Tell Computers and Humans Apart (CAPTCHAs), and delaying subsequent login attempts after several failed password attempts. Usually attackers will give up after a certain point due to the process taking too long and being too annoying for them to continue attempting.



Broken Authentication and Session Management



Broken Authentication and Session Management is a vulnerability defined by [OWASP](#)²⁷ which occurs when a pathway within an application is opened for an unauthorized person to access an authorized user's account, which may result in an attacker managing the session. Attackers know that software typically handles user information in a predictable way, allowing a human to anticipate how to work around the authentication system.

Secure software developers prevent this vulnerability by using caution when creating the authentication section of software. They avoid predictability in how sessions are named and handled, and carefully work to ensure authentication logic follows sane best practices. SSDLC developers must also work diligently to avoid duplicating authentication logic in order to better audit and maintain the software.

Insecure Direct Object References (IDOR)

An IDOR vulnerability is created whenever an application does not have enough security for user-supplied input, failing to prevent unauthorized users from accessing and viewing important databases or files. Software should function to allow only authorized users to access their information through the application, and significant problems can arise whenever the application fails to keep unauthorized users out.

Serious complications could arise from a user having their private information or private media accessed by an unauthorized entity. An IDOR vulnerability can happen over a variety of applications: web, mobile, desktop, and others.

IDOR Attacks and Sensitive Information: An Example

A health patient sends a private photo to their doctor over a web application. However, unbeknownst to anyone, an attacker is also

a user of the same web application. The attacker discovers that the system is not secure, and they are then able to view the patient's personal photo by simply changing the text in the URL from "Picture ID 12345" to "Picture ID 12346."

Sensitive Data Exposure

A Sensitive Data Exposure incident occurs when software leaks private information to people who should not see it, resulting in users' personal information being exposed. The result is often referred to as a data breach. These incidents have been highly prevalent in recent years and have cost businesses billions of dollars in revenue. A recent example of this attack occurred in 2017 when the popular wrestling entertainment company WWE stored three million users' data in plain-text on Amazon Web Services Simple Storage Service (S3). Likely due to human error, the data was misconfigured to be public instead of private. This granted anybody the ability to see each of the three million users' ethnicities, addresses, educational backgrounds, yearly earnings, and even their children's information. This oversight in web security left millions of users' sensitive and private data exposed to the world.



Missing Function Level Access Control

This software flaw occurs when a program attempts to process a sensitive operation, such as turning a normal user into an administrative user, without checking if the person requesting the operation is allowed to request it in the first place. This can be detrimental because administrative users may have special access within the application as well as normal user interface access. Secure software developers can prevent this flaw from occurring by diligently checking that every function with elevated access has specific security checks within the application to ensure that both normal users and administrative users can access only what they are allowed to access.

Cross-Site Request Forgery (CSRF)

Similar to a phishing attack, a cross-site request forgery uses social engineering to accomplish its goal of accessing private information. Typically, a user who is already securely logged into a system receives an email/Instant Message/Tweet/URL from an attacker who is pretending to be a trusted entity. The user clicks the link, believing that it is valid, and unknowingly performs an operation intended by the attacker in the target system. Because the link sent by the attacker asks the authorized user's browser to send the request to the system, the system cannot tell that the user was tricked into performing the illegitimate operation.

Using Known Vulnerable Components

This flaw in software development occurs whenever companies/developers unknowingly develop software in an insecure manner, usually by writing code that contains security vulnerabilities, and allow the code to be reused by others. When the nonsecure code is reused by others, the others unknowingly make their own software insecure by using the vulnerable component.

A common case of why this issue may arise happens whenever developers discover problems within a third-party component that they have been using for years. The developers decide that the problems are too expensive or too difficult to mitigate and choose to avoid solving the problem. This flaw is characterized by a lack of communication between the parties involved and a lack of awareness.

Unvalidated Redirects and Forwards

Unvalidated redirects and unvalidated forwards happens when a user goes to a URL that they believe will redirect/forward them to a good site, when in actuality they are unknowingly being redirected to a bad site that is disguised as the good site. These bad URLs can be sent by means of email or a link, similar to a phishing scam or a cross-site request forgery attack.

Next Steps

The staff of Digital Maelstrom regularly engages in security training and undergoes relevant security certifications to be qualified in secure practices, secure engineering, and penetration testing. Jeff Stice-Hall, the company's Principal Consultant for Security, has a number of industry certifications relevant to Security and Secure Software Development: including (ISC2) CSSLP (Certified Secure Software Lifecycle Professional), (ISC2) CISSP (Certified Information Systems Security Professional), Offensive Security OSCP (Offensive Security Certified Professional), and AWS Solutions Architect.

To speak with any of Digital Maelstrom's Secure Software Development professionals, see the Contact Us information at the bottom of this page.

About Digital Maelstrom

Digital Maelstrom is an IT advisory company which focuses on custom software, IT operations, IT security, and IT strategy. The name Digital Maelstrom represents the villain in the story of the hero who is buffeted on all sides by digital and technological chaos. DM's team is the trusted sage amidst this chaos who will help guide the hero back to their path and equip them with the tools to conquer the villain and accomplish their business goals.

Contact Us

Website: <https://www.digitalmaelstrom.net>

Phone: (816) 200-0245

Email: sales@digitalmaelstrom.net

Address: 1600 Genessee St Suite #528
Kansas City, Missouri, 64102



 [digitalmaelstrom](#)

 [digital-maelstrom-llc](#)

 [@DigitalMaelstrm](#)

 [@DigitalMaelstrom](#)

Endnotes

- 1 "Marriott Announces Starwood Guest Reservation Database Security Incident." Marriott News Center. November 30, 2018. Accessed April 2, 2019. <https://news.marriott.com/2018/11/marriott-announces-starwood-guest-reservation-database-security-incident/>.
- 2 "PCI Security." PCI Security Standards Council®. Accessed May 3, 2019. https://www.pcisecuritystandards.org/pci_security/.
- 3 "Cybersecurity Requirements for Financial Services Companies." New York State Dept. of Financial Services. Accessed April 2, 2019. <https://www.dfs.ny.gov/docs/legal/regulations/adoptions/dfsrf500txt.pdf>.
- 4 "Gramm-Leach-Bliley Act." Federal Trade Commission. Accessed May 21, 2019. <https://www.ftc.gov/tips-advice/business-center/privacy-and-security/gramm-leach-bliley-act>.
- 5 "Family Educational Rights and Privacy Act (FERPA)." Home. March 01, 2018. Accessed May 6, 2019. <https://www2.ed.gov/policy/gen/guid/fpc/fepa/index.html>.
- 6 HHS Office of the Secretary, Office for Civil Rights, and Ocr. "Summary of the HIPAA Privacy Rule." HHS.gov. July 26, 2013. Accessed May 06, 2019. <https://www.hhs.gov/hipaa/for-professionals/privacy/laws-regulations/index.html>.
- 7 "Federal Information Security Modernization Act." Department of Homeland Security. April 29, 2019. Accessed May 6, 2019. <https://www.dhs.gov/cisa/federal-information-security-modernization-act>.
- 8 "Assembly Bill No. 375 Chapter 55." California Legislative Information. June 29, 2018. Accessed May 06, 2019. https://leginfo.ca.gov/faces/billTextClient.xhtml?bill_id=201720180AB375.
- 9 "The EU General Data Protection Regulation (GDPR) Is the Most Important Change in Data Privacy Regulation in 20 Years." EUGDPR.org. Accessed May 21, 2019. <https://eugdpr.org/>.
- 10 "NIS Directive." ENISA. January 18, 2019. Accessed May 21, 2019. <https://www.enisa.europa.eu/topics/nis-directive>.
- 11 Oxley, and Michael G. "H.R.3763 - 107th Congress (2001-2002): Sarbanes-Oxley Act of 2002." Congress.gov. July 30, 2002. Accessed May 21,

2019. <https://www.congress.gov/bill/107th-congress/house-bill/3763>.

12 “Who Is Liable When a Data Breach Occurs?” Thomson Reuters Legal. Accessed May 21, 2019. <https://legal.thomsonreuters.com/en/insights/articles/data-breach-liability>.

13 <https://www.ftc.gov/enforcement/cases-proceedings/refunds/equifax-data-breach-settlement>

14 <https://www.nytimes.com/2017/10/03/technology/yahoo-hack-3-billion-users.html>

15 “2017 State of Cybersecurity in Small & Medium-Sized Businesses (SMB).” CSRPS. September 2017. Accessed May 21, 2019. <https://www.csrps.com/wp-content/uploads/2019/03/2017-Ponemon-State-of-Cybersecurity-in-Small-and-Medium-Sized-Businesses-SMB.pdf>.

16 [https://en.wikipedia.org/wiki/STRIDE_\(security\)](https://en.wikipedia.org/wiki/STRIDE_(security))

17 [https://en.wikipedia.org/wiki/Defense_in_depth_\(computing\)](https://en.wikipedia.org/wiki/Defense_in_depth_(computing))

18 SONARQUBE is trademark of SonarSource SA.

19 OWASP Top Ten Items of 2017 by OWASP is released under Creative Commons Attribution Share-Alike 4.0

license.

20 OWASP Top 10 - 2017, 12.

21 <https://techcrunch.com/2015/10/05/patreon-hacked-gigabytes-of-data-and-code-leaked/>

22 OWASP Top 10 - 17, 13.

23 “Code Injection.” OWASP. December 31, 2013. Accessed May 6, 2019. https://www.owasp.org/index.php/Code_Injection.

24 OWASP Top 10 - 17, 13.

25 Ibid.

26 OWASP Top 10 - 2017, 6.

27 https://www.owasp.org/index.php/Broken_Authentication_and_Session_Management



www.digitalmaelstrom.net

All rights reserved (c) 2019 Digital Maelstrom